

Robot Calibration Library

ロボットアームのキャリブレーションを行う Python ライブラリ。
DH パラメータ誤差・ツール変換誤差・ベース座標系誤差・時刻ずれを最小二乗法で同定します。
ラプラス近似による不確かさ評価と Before/After サマリーグラフ出力に対応しています。

アーキテクチャ

実験ごとに **レシピファイル** (`recipe_*.py`) を 1 つ作成するだけで実験が完結します。
骨格コード (`estimation/`, `io/`, `visualization/`) は一切触りません。

```
calib/
├─ recipe_ur5_calibration.py      ← ランダム軌道からの同定例
├─ recipe_straight_line_calib.py ← 直線軌道 + 時刻ずれ同定例
└─ robot_calibration/
    ├─ pipeline.py                ← run_calibration / compute_uncertainty
    ├─ models/
    │   ├─ base.py                ← KinematicModel / ObservationModel (ABC)
    │   └─ defaults.py           ← DHKinematics / PoseObservation /
DistanceObservation
    │   ├─ parameters.py          ← Parameter / ParameterSet
    │   └─ transforms.py         ← ObservationTransform 各種
    ├─ estimation/
    │   ├─ optimizer.py           ← Stage / StageResult
    │   └─ uncertainty.py         ← laplace_uncertainty / UncertaintyResult
    ├─ visualization/
    │   └─ plotter.py             ← plot_calibration_summary など
    └─ io/
        └─ loader.py             ← CSV 読み込み / 保存
```

インストール

```
conda env create -f environment.yml
conda activate calib
```

依存関係: Python >= 3.11, NumPy, SciPy, Matplotlib

典型的なワークフロー

```
import numpy as np
from robot_calibration import (
    run_calibration, compute_uncertainty,
```

```

    DHKinematics, PoseObservation, Parameter, Stage,
)
from robot_calibration.models.transforms import IdentityTransform
from robot_calibration.visualization.plotter import plot_calibration_summary
from robot_calibration.io.loader import load_joint_trajectory,
load_position_observations

# 1. ロボット・モデル定義
DH_NOMINAL = [...] # Modified DH パラメータ (6 リンク分)
KIN = DHKinematics(DH_NOMINAL)
OBS = PoseObservation()

# 2. 推定パラメータ定義
PARAMETERS = [
    *[Parameter(f"d_alpha_{i}", value=0.0, group="kinematic", prior_std=np.pi) for
i in range(6)],
    *[Parameter(f"d_a_{i}", value=0.0, group="kinematic", prior_std=1.0) for
i in range(6)],
    *[Parameter(f"d_d_{i}", value=0.0, group="kinematic", prior_std=1.0) for
i in range(6)],
    *[Parameter(f"d_theta_offset_{i}", value=0.0, group="kinematic",
prior_std=np.pi) for i in range(6)],
    Parameter("tool_tx", value=0.0, group="tool", prior_std=1.0),
    Parameter("tool_ty", value=0.0, group="tool", prior_std=1.0),
    Parameter("tool_tz", value=0.0, group="tool", prior_std=1.0),
    Parameter("tool_rx", value=0.0, group="tool", prior_std=np.pi),
    Parameter("tool_ry", value=0.0, group="tool", prior_std=np.pi),
    Parameter("tool_rz", value=0.0, group="tool", prior_std=np.pi, fixed=True),
# 縮退 → 固定
    Parameter("local_tx", value=0.0, group="local", prior_std=1.0),
    Parameter("local_ty", value=0.0, group="local", prior_std=1.0),
    Parameter("local_tz", value=0.0, group="local", prior_std=1.0),
    Parameter("local_rx", value=0.0, group="local", prior_std=np.pi),
    Parameter("local_ry", value=0.0, group="local", prior_std=np.pi),
    Parameter("local_rz", value=0.0, group="local", prior_std=np.pi),
]

# 3. データ読み込み
q_times, q_traj = load_joint_trajectory("joint_angles.csv") # (N,), (N, 6)
_, y_exp = load_position_observations("tcp_positions.csv") # (N,), (N, 3)

# 4. 推定実行
STAGES = [Stage("calib", param_groups=["kinematic", "tool", "local"],
transform=IdentityTransform())]
params_result, stage_results = run_calibration(
    q_traj=q_traj, y_exp=y_exp,
    parameters=PARAMETERS, kinematic_model=KIN, observation_model=OBS,
    stages=STAGES,
)
print(params_result.summary())

# 5. 不確かさ評価 (ラプラス近似)
uncertainty = compute_uncertainty(
    params=params_result, kin_model=KIN, obs_model=OBS,

```

```

    q_traj=q_traj, y_exp=y_exp,
)
print(uncertainty.summary())    # 各パラメータの推定値 ± 1σ

# 6. Before/After サマリーグラフ出力
calib_dict    = {p.name: p.value for p in params_result.params}
nominal_dict  = {p.name: 0.0    for p in params_result.params}
p_before = np.array([OBS.predict(KIN.forward(q_traj[i], nominal_dict), {}) for i
in range(len(q_traj))])
p_after  = np.array([OBS.predict(KIN.forward(q_traj[i], calib_dict),   {}) for i
in range(len(q_traj))])

fig = plot_calibration_summary(
    p_exp=y_exp, p_pred_before=p_before, p_pred_after=p_after,
    uncertainty=uncertainty,
)
fig.savefig("output/summary.png", dpi=150)

```

サンプルレシピ

ファイル	内容
recipe_ur5_calibration.py	ランダム姿勢 100 点からの DH / ツール / ベース誤差同定
recipe_straight_line_calib.py	直線軌道 (500 Hz 高レート) + 時刻ずれ同定

```

python recipe_ur5_calibration.py
python recipe_straight_line_calib.py

```

いずれも **output/** ディレクトリに Before/After サマリーグラフを保存します。

run_calibration API

```

params_result, stage_results = run_calibration(
    q_traj          : np.ndarray,          # (N, n_joints) 関節角度 [rad]
    y_exp           : np.ndarray,          # (N, obs_dim) または
    (N*obs_dim,) 観測値
    parameters      : list[Parameter],     # 推定パラメータリスト
    kinematic_model : KinematicModel,      # 順運動学モデル
    observation_model : ObservationModel,   # 観測モデル
    stages          : list[Stage],         # 推定ステージリスト
    ls_kwargs       : dict | None = None,  # scipy.optimize.least_squares
    への追加引数
    q_timestamps    : np.ndarray | None = None, # タイムスタンプ (N,) -
    time_offset     : 推定時に必要
    final_full_tune : bool = False,        # 全ステージ後に全パラメータで最終

```

調整

```
) -> tuple[ParameterSet, list[StageResult]]
```

compute_uncertainty API

```
uncertainty = compute_uncertainty(
    params          : ParameterSet,          # run_calibration の戻り値
    kin_model       : KinematicModel,
    obs_model       : ObservationModel,
    q_traj          : np.ndarray,
    y_exp           : np.ndarray,
    transform       : ObservationTransform | None = None, # 省略時
    IdentityTransform
    q_timestamps    : np.ndarray | None = None,
) -> UncertaintyResult # .param_names, .means, .stds, .cov
```

ヤコビアン J を最適解で評価し、 $Cov \approx (J^T J)^{-1}$ からパラメータ標準偏差を計算します。
 σ が大きいパラメータは軌道から同定しにくい（情報不足・縮退に近い）ことを意味します。

パラメータ定義

```
Parameter(
    name      : str,          # DHKinematics が認識するキー名と対応
    value     : float,        # 初期値
    group     : str,          # "kinematic" | "tool" | "local" | "time_offset" |
    任意
    prior_std : float,        # 事前分布の標準偏差（大きいほど非情報の）
    fixed     : bool = False, # True にすると推定から除外
)
```

DHKinematics が認識するパラメータ名

グループ	パラメータ名	意味
kinematic	d_alpha_i, d_a_i, d_d_i, d_theta_offset_i	DH パラメータ誤差 (i=0..n-1)
tool	tool_tx/ty/tz [m], tool_rx/ry/rz [rad]	ツール変換誤差
local	local_tx/ty/tz [m], local_rx/ry/rz [rad]	ベース座標系誤差
time_offset	time_offset [s]	制御と観測の時刻ずれ

縮退パラメータの扱い

位置のみ観測の場合、tool_rz と d_theta_offset_5（最終軸まわり回転）は完全縮退で分離不能です。
 慣例として tool_rz を fixed=True に設定し、効果を d_theta_offset_5 に吸収させます。

時刻ずれ推定 (time_offset)

ロボットコントローラと外部計測器のクロックずれを同定します。

`q_timestamps` を `run_calibration` に渡すと、パイプライン内で cubic 補間を使い

$q_{\text{eff}}(t) = q(t + \text{time_offset})$ として FK を評価します。

```
# パラメータに time_offset を追加
PARAMETERS += [Parameter("time_offset", value=0.0, group="time_offset",
prior_std=1.0)]

# 2 ステージ推定 (先に time_offset、次に DH 誤差)
STAGES = [
    Stage("stage1_time_offset",
          param_groups=["time_offset"],
          transform=IdentityTransform()), # 位置残差は time_offset に対して単調 →
    確実に収束
    Stage("stage2_kinematics",
          param_groups=["kinematic", "tool", "local"],
          transform=IdentityTransform()),
]

params_result, _ = run_calibration(
    ..., stages=STAGES, q_timestamps=q_timestamps,
)
```

注意: `q_timestamps` に渡す関節角度時系列は **C1 連続** (速度連続) である必要があります。
 ライン間に不連続な停止区間を挿入すると cubic 補間がスパイクを生じます。
 ライン間は smooth-step 関数などで関節空間ブレンドしてください ([recipe_straight_line_calib.py](#) 参照)。

推定ステージ定義

```
from robot_calibration import Stage
from robot_calibration.models.transforms import IdentityTransform

Stage(
    name          : str,                # ステージ名 (ログ表示用)
    param_groups  : list[str] | None,   # 推定対象グループ (None = 全パラメータ)
    transform     : ObservationTransform,
)
```

観測変換

クラス	用途
<code>IdentityTransform</code>	位置残差を直接使用 (汎用・ <code>time_offset</code> 推定にも適用可)

クラス	用途
<code>VelocityNormTransform(dt)</code>	速度ノルム時系列に変換。単純な 1 軸回転運動での <code>time_offset</code> 推定に有効
<code>FFTAplitudeTransform</code>	残差ノルムの FFT 振幅。周期的伝達誤差推定に有効
<code>IdentityTransform</code> は位置残差の勾配が <code>time_offset</code> に対して単調なため、多関節 3D 軌道での <code>time_offset</code> 推定に最も安定して収束します。	

Before/After 可視化

```
from robot_calibration.visualization.plotter import plot_calibration_summary

fig = plot_calibration_summary(
    p_exp          : np.ndarray,          # 観測位置 (N, 3) [m]
    p_pred_before  : np.ndarray,          # キャリブ前予測 (N, 3) [m]
    p_pred_after   : np.ndarray,          # キャリブ後予測 (N, 3) [m]
    uncertainty    : UncertaintyResult | None = None, # compute_uncertainty() の
    戻り値
    true_errors    : dict[str, float] | None = None, # シミュレーション時の真値
    title          : str = "Calibration Summary",
) -> matplotlib.figure.Figure
```

生成されるグラフ (3 段構成) :

段	内容
上段	XYZ 各軸の残差時系列 (Before/After 重ね描き) + XY 誤差散布図
中段	点ごとの誤差ノルム時系列 + RMS 改善率棒グラフ
下段	パラメータ推定値 $\pm 1\sigma$ (真値がある場合は $\times$ で重ねて表示)

カスタムモデルの実装

`DHKinematics` や `PoseObservation` では対応できない場合はサブクラス化します。

```
from robot_calibration import KinematicModel, ObservationModel
import numpy as np

class MyKinematics(KinematicModel):
    def forward(self, q: np.ndarray, params: dict) -> np.ndarray:
        # params は {名前: 値} の辞書。未定義キーは .get(key, 0.0) で取る
        return T # (4, 4) SE(3)

    def jacobian(self, q: np.ndarray, params: dict) -> np.ndarray:
        return J # (3, n_joints)
```

```
class MyObservation(ObservationModel):
    def predict(self, x: np.ndarray, params: dict) -> np.ndarray:
        return x[:3, 3] # 例: 3D 位置
```

`KinematicModel` には `numerical_jacobian()` が実装済みで、解析的ヤコビアンを検証に使えます。

データフォーマット

関節角度軌道 (`joint_angles.csv`)

```
t,q0,q1,q2,q3,q4,q5
0.000,0.1,0.2,0.3,0.4,0.5,0.6
0.002,0.11,0.21,0.31,0.41,0.51,0.61
```

TCP 位置観測 (`tcp_positions.csv`)

```
t,px,py,pz
0.000,0.412,0.031,0.553
0.002,0.413,0.031,0.554
```

```
from robot_calibration.io.loader import load_joint_trajectory,
load_position_observations

q_times, q_traj = load_joint_trajectory("joint_angles.csv") # → (N,), (N, 6)
_, y_exp = load_position_observations("tcp_positions.csv") # → (N,), (N, 3)
```

テスト実行

```
python -m robot_calibration.tests.simulation_test
```

テスト名	内容
ヤコビアン検証	解析的ヤコビアンと数値微分の一致確認
DH ヤコビアン検証	DH パラメータ誤差ヤコビアンの確認
伝達誤差同定	周期的関節誤差の同定
<code>test_trajectory_identification</code>	6 関節サイン波軌道 + 時刻ずれ同定 (20 ms 精度 < 5 ms)
<code>test_straight_line_identification</code>	直線軌道 9 本から DH / ツール / ベース誤差同定

テスト成功時、プロット画像が `robot_calibration/tests/output/` に保存されます。

API リファレンス

`robot_calibration` トップレベルエクスポート

名前	種別	説明
<code>run_calibration</code>	関数	統合キャリブレーションパイプライン
<code>compute_uncertainty</code>	関数	ラプラス近似による不確かさ評価
<code>KinematicModel</code>	ABC	順運動学モデルのインタフェース
<code>ObservationModel</code>	ABC	観測モデルのインタフェース
<code>DHKinematics</code>	クラス	Modified DH パラメータによる順運動学
<code>PoseObservation</code>	クラス	3D 位置観測モデル
<code>DistanceObservation</code>	クラス	固定点からの距離スカラー観測モデル
<code>Parameter</code>	クラス	推定パラメータ (name, value, group, prior_std, fixed)
<code>ParameterSet</code>	クラス	パラメータコレクション (<code>.summary()</code> , <code>.free_indices()</code> など)
<code>Stage</code>	クラス	推定ステージ定義 (name, param_groups, transform)
<code>StageResult</code>	クラス	ステージ結果 (cost, success, message, jacobian)
<code>UncertaintyResult</code>	クラス	不確かさ評価結果 (param_names, means, stds, cov)